What Evidence Shows That Software Is Post-Quantum Secure? A Layered Framework for Assessing Post-Quantum Cryptographic Security in Software

To decide whether software is post-quantum (PQ) cryptographically secure, you need verifiable evidence on four layers: algorithms, protocols, implementation and operations. That evidence starts with a complete cryptographic inventory (CBOM) and shows that every quantum-vulnerable public-key use (RSA, finite-field DH, ECDH, ECDSA and EdDSA) has been replaced by, or hybridized with, a final NIST PQC standard (FIPS 203, 204 or 205, or SP 800-208 LMS/XMSS). The evidence must also show that the PQ mechanism is actually negotiated in practice, that it is implemented by a validated and side-channel-hardened module, and that the system can change algorithms again.

TL;DR

- Treat "PQ-secure" as a graded, layered property. No single test proves it. The verdict depends on four kinds of evidence: (1) a complete CBOM; (2) proof that PQ or hybrid key establishment and signatures are actually *negotiated* in practice, not merely supported; (3) implementation assurance such as FIPS 140-3/CAVP validation, constant-time evidence, test vectors and CVE history; and (4) crypto-agility and downgrade resistance, weighed against data lifetime using Mosca's inequality.
- As of October 2026 the standards base is mature for key establishment but not for authentication. FIPS 203/204/205 (August 2024), SP 800-227 (September 2025), RFC 10024 (hybrid ML-KEM for TLS 1.3, August 2026) and RFC 10042 (hybrid ML-KEM for SSH, August 2026) are final. nist +3 FIPS 206 (FN-DSA), the HQC standard and NIST IR 8547's 2030/2035 dates are still unfinished, and PQ certificates in the Web PKI remain largely undeployed. neuraparse +3
- Deadlines have hardened. US EO 14412 (22 June 2026) requires PQ key establishment for federal high-value and high-impact systems by 31 December 2030 and PQ signatures by 31 December 2031. dell bostonidentity CNSA 2.0, the EU roadmap, the UK NCSC, Canada and Australia all converge on 2030–2035. Assessors should therefore prioritize key-establishment evidence now and signature/PKI evidence next.

Abstract

Large-scale fault-tolerant quantum computers would break the public-key cryptography that secures almost all deployed software. NIST finalized its first post-quantum cryptography (PQC) standards in August 2024, and several governments converted migration guidance into dated obligations in 2025–2026. Software owners, auditors and procurers therefore face a practical question: *what information must be gathered to determine whether a given piece of*

software is post-quantum cryptographically secure? This paper answers that question for both white-box assessment (source code, dependencies, builds and configuration of software one develops) and black-box or grey-box assessment (third-party software one procures or audits). We first define PQ security as a layered property that spans algorithms, protocols, implementations and operations, and we explain why key establishment and authentication carry different urgency. We then survey the standards and regulatory landscape as of October 2026, distinguishing final standards from drafts. The core contribution is a catalogue of information requirements, each with its rationale, collection method and acceptance criteria. It is followed by an analysis of evidence-collection methods and their false-negative risks, a six-stage assessment framework with a five-level readiness scale, and a worked case study of a web service with an OpenSSH estate and a firmware-update pipeline. We conclude that no formal "PQ-secure" certification for software yet exists. Defensible verdicts must therefore be evidence-based, scoped to a protocol and use case, time-bound, and re-evaluated as standards (notably FIPS 206, the HQC standard and NIST IR 8547) are finalized.

Keywords: post-quantum cryptography; ML-KEM; ML-DSA; SLH-DSA; cryptographic inventory; cryptography bill of materials (CBOM); crypto-agility; hybrid key exchange; TLS 1.3; software assurance; quantum risk assessment; migration.

---

1. Introduction

1.1 Motivation

Public-key cryptography based on integer factorization and discrete logarithms underpins TLS, SSH, IPsec, code signing, secure boot, PKI, secure messaging and most authentication systems. Shor's algorithm would break all of these schemes on a sufficiently large fault-tolerant quantum computer, known as a cryptographically relevant quantum computer (CRQC) [1]. The estimated cost of building such a machine has fallen steeply. In 2019, Gidney and Ekerå estimated that RSA-2048 could be factored in about eight hours with 20 million noisy qubits [2]. postquantum arxiv In May 2025, Gidney reduced this to under one million noisy qubits in under a week, under the same hardware assumptions (0.1% gate error, 1 μs surface-code cycle, 10 μs reaction time) [3]. postquantum research In February 2026, the Pinnacle Architecture preprint from Iceberg Quantum claimed that fewer than 100,000 physical qubits would suffice using quantum LDPC codes, at a runtime of about a month [4]. alphaxiv +5 These last results are preprints and rest on architectural assumptions that have not been demonstrated at scale. thequantuminsider Even so, the trend is unambiguous and has shifted expert opinion. The Global Risk Institute's 2025 Quantum Threat Timeline Report, published in March 2026, surveyed 26 experts. They judged a CRQC "quite possible" (28–49%) within 10 years and "likely" (51–70%) within 15 years [5]. globalriskinstitute +3

Policy has moved accordingly. NIST published FIPS 203 (ML-KEM), FIPS 204 (ML-DSA) and FIPS 205 (SLH-DSA) on 13 August 2024 [6]–[8]. openssl-corporation iamuvin On 22 June 2026,

US Executive Order 14412 set binding federal dates: PQ key establishment for high-value assets and high-impact systems by 31 December 2030, and PQ digital signatures by 31 December 2031 [9]. arcqubit OMB Memorandum M-26-15 operationalized these dates two days later [10]. openssl-corporation zscaler The European Union, the United Kingdom, Canada and Australia have published comparable timelines, examined in Section 4. openssl-corporation

Organizations therefore need to answer, at scale and with evidence, whether each piece of software they build or buy is PQ-secure. This is harder than it looks. Cryptography is buried in transitive dependencies, platform libraries, hardware modules, configuration files and vendor black boxes. “Supports ML-KEM” in a datasheet does not mean that ML-KEM is negotiated in production, let alone that it is implemented without timing leaks.

1.2 Research question

What information (evidence, artifacts, data points and criteria) must be collected and evaluated to determine whether a given piece of software is post-quantum cryptographically secure, i.e., whether its cryptography resists both a CRQC and classical adversaries?

1.3 Contributions

1. A **layered definition** of PQ-secure software (algorithm, protocol, implementation, system/operations) that replaces a binary verdict with a scoped, graded one (Section 3).
2. A **status-verified survey** of standards and government timelines as of October 2026, explicitly marking final versus draft items and recent changes (Section 4).
3. A **master catalogue of information requirements** with rationale, collection method and acceptance criteria (Section 5, Table 4).
4. An analysis of **evidence-collection methods** and their characteristic false negatives (Section 6).
5. A **six-stage assessment framework**, decision matrix and five-level readiness scale (Section 7), demonstrated in a **worked case study** (Section 8).

---

2. Background

2.1 Quantum algorithms and their cryptographic impact

Shor’s algorithm solves integer factorization and discrete logarithms (in finite fields and on elliptic curves) in polynomial time [1]. It therefore breaks RSA (encryption and signatures), finite-field Diffie–Hellman (DH/DSA), elliptic-curve Diffie–Hellman (ECDH, X25519), ECDSA and EdDSA. Larger key sizes do not help: the attack scales polynomially, so moving from RSA-2048 to RSA-4096 buys only a constant factor.

Grover’s algorithm gives a quadratic speedup for unstructured search. In principle this halves the effective bit-security of a symmetric key against exhaustive search. In practice, Grover’s search parallelizes poorly and requires long sequential runs of error-corrected operations.

NIST therefore assesses that AES-128 remains adequate and defines its PQC security categories relative to the cost of key search on AES-128, AES-192 and AES-256 (Categories 1, 3, 5) and collision search on SHA-256/SHA-384 (Categories 2, 4) [6]. NIST's draft transition report IR 8547 does not propose deprecating AES-128 on quantum grounds [11]. The NSA's CNSA 2.0 suite is more conservative for National Security Systems. It mandates AES-256 for symmetric encryption and SHA-384 or SHA-512 for hashing [12]. intellixbom For hash functions, quantum collision-finding gives little practical advantage over classical attacks, so SHA-256 and SHA-3 remain sound for general use. An assessor should record which of these two positions (NIST general-purpose or CNSA 2.0) the software is required to meet.

2.2 Resource estimates and timelines

Table 1 summarizes the evolution of resource estimates for breaking RSA-2048.

Table 1. Selected resource estimates for factoring RSA-2048 on a fault-tolerant quantum computer.

Source	Year	Physical qubits	Runtime	Status / key assumption
Gidney & Ekerå [2]	2019 (Quantum, 2021)	~20 million	~8 hours	Peer-reviewed; surface code, 10^{-3} error rate
Gidney [3]	May 2025	< 1 million (~898k reported)	< 1 week	Preprint; same physical assumptions as [2]
Webster et al., Pinnacle [4]	Feb 2026	< 100,000	~1 month	Preprint; quantum LDPC codes with non-local connectivity

The planning implication is not a date but a *distribution*. Expert surveys [5] place substantial probability on a CRQC within the planning horizon of systems being deployed today.

2.3 Harvest-now-decrypt-later and trust-now-forge-later

Two threat models determine urgency. **Harvest now, decrypt later (HNDL)** applies to confidentiality. An adversary records encrypted traffic or stored ciphertext today and decrypts it once a CRQC exists. Any session whose key establishment uses ECDH or RSA key transport is exposed *retroactively*, so the risk exists now for data with a long confidentiality lifetime. **Trust now, forge later** applies to authentication. Signatures cannot be forged retroactively to affect past sessions. However, long-lived trust anchors (root CA keys, firmware-signing keys burned into ROM, long-validity code-signing certificates) will remain in service when a CRQC arrives, and the adversary can then forge signatures under them. Devices that cannot rotate their roots of trust are especially exposed.

2.4 Mosca's inequality

Mosca's framing gives a simple decision rule. Let x be the time data must remain confidential (or a signature must remain trustworthy), y the time needed to migrate the system, and z the time until a CRQC exists. If $x + y > z$, the organization is already exposed [5]. The inequality directly determines which information an assessment needs: data shelf-life, system and device service life, and migration lead time (Section 5.8).

2.5 Families of post-quantum cryptography

- **Lattice-based** (Module-LWE / Module-SIS, NTRU): ML-KEM (FIPS 203), ML-DSA (FIPS 204), FN-DSA (draft FIPS 206, from Falcon). These offer good performance with moderate key and ciphertext sizes.
- **Hash-based signatures**: stateless SLH-DSA (FIPS 205, from SPHINCS+), and stateful LMS/HSS and XMSS/XMSS^MT (RFC 8554, RFC 8391, NIST SP 800-208). Their security rests only on hash-function properties, which makes them very conservative, at the cost of large signatures (SLH-DSA) or state management (LMS/XMSS).
- **Code-based**: HQC, selected by NIST in March 2025 as a backup KEM to ML-KEM [13], securityweek arcqubit and Classic McEliece, which is not NIST-selected but is used in some national contexts.
- **Others**: multivariate (UOV, MAYO and others), isogeny-based (SQIsign) and MPC-in-the-head schemes such as FAEST, MQOM and SDitH. These are now third-round candidates in NIST's additional-signature process [14].

Table 2. Quantum-vulnerable vs. quantum-resistant public-key algorithms (sizes in bytes).

Algorithm	Type	Quantum status	NIST category	Public key	Ciphertext / signature	Standard status (Oct 2026)
RSA-2048	Sig / key transport	Broken by Shor	— (≈112-bit classical)	256 (modulus)	256	FIPS 186-5; proposed deprecation after 2030 [11]
ECDSA P-256 / ECDH P-256	Sig / KEX	Broken by Shor	— (≈128-bit classical)	64 (65 uncompressed)	64 (sig)	Proposed disallowance after 2035 [11]
X25519 / Ed25519	KEX / Sig	Broken by Shor	—	32	32 (share) / 64 (sig)	As above
ML-KEM-512	KEM	Resistant	1	800	768	Final, FIPS 203 (Aug 2024)
ML-KEM-768	KEM	Resistant	3	1,184	1,088	Final, FIPS 203
ML-KEM-1024	KEM	Resistant	5	1,568	1,568	Final, FIPS 203; CNSA 2.0
ML-DSA-44	Signature	Resistant	2	1,312	2,420	Final, FIPS 204 (Aug 2024)
ML-DSA-65	Signature	Resistant	3	1,952	3,309	Final, FIPS 204
ML-DSA-87	Signature	Resistant	5	2,592	4,627	Final, FIPS 204; CNSA 2.0
SLH-DSA-SHA2-128s / -128f	Signature	Resistant	1	32	7,856 / 17,088	Final, FIPS 205 (Aug 2024)
SLH-DSA-SHA2-192s / -192f	Signature	Resistant	3	48	16,224 / 35,664	Final, FIPS 205
SLH-DSA-SHA2-256s	Signature	Resistant	5	64	29,792 / 49,856	Final, FIPS 205

Algorithm	Type	Quantum status	NIST category	Public key	Ciphertext / signature	Standard status (Oct 2026)
/ -256f						
FN-DSA-512 / -1024	Signature	Resistant	1 / 5	897 / 1,793	~666 / ~1,280	**Not final**, FIPS 206 in development
HQC-128 / -192 / -256	KEM	Resistant	1 / 3 / 5	2,249 / 4,522 / 7,245	4,433 / 8,978 / 14,421	**Not final**, selected Mar 2025; draft pending
LMS / XMSS (SP 800-208)	Stateful sig	Resistant	parameter-dependent	~32–64	~1–9 KB (parameter-dependent)	Final, SP 800-208 (Oct 2020); CNSA 2.0 for firmware

Notes: ML-KEM, ML-DSA and SLH-DSA sizes are from FIPS 203/204/205 [6]–[8]; RFC 9935 reproduces the ML-KEM figures [15]. FN-DSA and HQC sizes are from the Falcon and HQC submission specifications and may change in the final standards.

3. Defining "PQ-Secure Software"

A binary label ("PQ-secure: yes/no") is misleading, because a single application may use a dozen cryptographic mechanisms with different risk profiles. We define PQ security as the conjunction of properties at four layers, each assessed per *use case* (for example "TLS server authentication on the public API" or "firmware update signature verification").

3.1 Algorithm layer

The software uses only algorithms that resist known quantum and classical attacks, at parameter sets meeting its required NIST security category. Concretely:

- every key-establishment use employs ML-KEM, alone or in a hybrid;
- every signature use employs ML-DSA, SLH-DSA or LMS/XMSS (with FN-DSA once FIPS 206 is final);
- symmetric primitives and hashes meet the applicable bar (AES-128+/SHA-256+ under NIST; AES-256/SHA-384+ under CNSA 2.0).

NIST categories 1–5 provide the common yardstick. CNSA 2.0 requires Category 5 parameters (ML-KEM-1024, ML-DSA-87) for all classification levels [12].

3.2 Protocol layer

The algorithms must be bound into protocols by a standardized, analyzed construction:

- TLS 1.3 hybrid groups (RFC 10024) [16];
- SSH hybrid key exchange (RFC 10042) [17];
- IKEv2 multiple key exchanges (RFC 9370) [18];
- X.509 with ML-DSA (RFC 9881), SLH-DSA (RFC 9909) and ML-KEM (RFC 9935) [15], [19], [20];
- CMS with KEMs (RFC 9629, RFC 9936) [21], [22];
- messaging protocols such as Signal PQXDH and its Sparse Post-Quantum Ratchet [23], [24].

Ad hoc constructions such as home-grown KEM combiners or custom message formats are a red flag, because their security depends on unreviewed analysis.

3.3 Implementation layer

The code that executes these algorithms must be:

- correct (passes known-answer and edge-case tests);
- constant-time with respect to secrets;
- resistant to faults where the threat model includes physical access;
- compliant with mandatory input checks (FIPS 203 requires encapsulation-key and decapsulation-key checks before use [6]);
- seeded by an approved DRBG (Deterministic Random Bit Generator).

Independent validation, through CAVP algorithm certificates and FIPS 140-3 module certificates, is the strongest available external evidence.

3.4 System and operational layer

The deployment must actually *use* the PQ mechanism, through configuration, negotiation and default settings. It must resist downgrade to classical-only modes and manage PQ keys through their lifecycle (generation, storage in HSM/TPM/KMS, rotation, revocation, and state for LMS/XMSS). It must also be *crypto-agile*: able to change algorithms again without redesign, as NIST CSWP 39 describes [25].

3.5 Differential urgency: confidentiality versus authentication

Because of HNDL, *key establishment* is the urgent layer. Every day of classical-only key exchange adds to an adversary's archive. Authentication is less urgent for ephemeral sessions

but urgent for long-lived roots of trust, particularly firmware and code signing, where CNSA 2.0 sets its earliest exclusive-use date (2030) [12]. EO 14412 encodes the same priority ordering: key establishment by 2030, signatures by 2031 [9].

3.6 Hybrid versus pure PQ

A *hybrid* (PQ/T) scheme combines a traditional and a PQ algorithm so that the result is secure if *either* component is secure (terminology per RFC 9794 [26]). European agencies (BSI, ANSSI, and the EU roadmap) recommend hybrids during the transition, as a hedge against cryptanalytic or implementation failures in young PQ schemes [27], [28]. The NSA's CNSA 2.0 allows hybrids where protocols require them but does not require them, and targets pure PQ at Category 5 for National Security Systems [12]. A defensible assessment does not prescribe one answer universally. It records which posture the applicable regime requires and verifies that the chosen construction is a standardized combiner.

Working definition. *Software is PQ-secure for a scoped use case when every quantum-vulnerable public-key operation in that use case has been replaced by, or hybridized with, an approved PQ algorithm at the required security category; when that mechanism is bound by a standardized protocol construction, implemented by an assured implementation, and actually negotiated in operation without exploitable downgrade; and when the system retains the agility to change these choices.* "Fully PQ-secure software" is the conjunction across all in-scope use cases.

---

4. Standards and Regulatory Landscape (status as of October 2026)

4.1 NIST algorithm standards and guidance

Table 3a. NIST PQC publications: status.

Document	Subject	Status (Oct 2026)	Notes
FIPS 203	ML-KEM	**Final**, 13 Aug 2024 [6]	Parameter sets 512/768/1024
FIPS 204	ML-DSA	**Final**, 13 Aug 2024 [7]	Pure and pre-hash (HashML-DSA) variants
FIPS 205	SLH-DSA	**Final**, 13 Aug 2024 [8]	12 parameter sets (SHA2/SHAKE, s/f)
FIPS 206	FN-DSA (Falcon)	**In development; not final**	NIST described the initial public draft as "basically written, awaiting approval" in Aug 2025. Secondary sources conflict on whether a public draft has appeared; final expected late 2026–2027
HQC standard (presented as "FIPS 207")	HQC-KEM	**Selected 11 Mar 2025 (NIST IR 8545); draft pending** [13]	NIST presented the forthcoming standard at its Sixth PQC Standardization Conference (Sept 2026) and signaled a seed-only private-key format; final expected 2027
Additional signatures	On-ramp	**Third round**: NIST IR 8610, 14 May 2026 [14]	FAEST, HAWK, MAYO, MQOM, QR-UOV, SDitH, SNOVA, SQIsign, UOV; ~2-year round
SP 800-208	LMS/HSS, XMSS/XMSS^MT	**Final**, Oct 2020 [29]	Requires state management; CNSA 2.0 for firmware
SP 800-227	KEM recommendations	**Final**, 18 Sept 2025 [30]	Draft Jan 2025
NIST IR 8547	Transition timeline	**Initial public draft** (12 Nov 2024) [11]	Proposes deprecating quantum-vulnerable algorithms (112-bit) after 2030 and disallowing them after 2035. Still listed as draft in Aug 2026. Some commercial trackers wrongly report it as final

Document	Subject	Status (Oct 2026)	Notes
CSWP 39	Crypto agility	**Final** 19 Dec 2025; **updated (39upd1) 29 Jun 2026** [25]	Defines crypto-agility capabilities and strategies
SP 1800-38A/B/C	NCCoE Migration to PQC	Preliminary drafts (multi-volume practice guide) [31]	Inventory/discovery tooling, interoperability and performance testing

Two status points need emphasis. First, IR 8547's 2030/2035 dates remain a *proposal*. They have nonetheless become the working baseline, because OMB M-26-15 directs agencies to align with them and EO 14412 imposes hard 2030/2031 dates on the highest-risk federal systems [9], [10]. Second, FN-DSA cannot yet be cited as an approved standard, so evidence that software "uses FIPS 206" should be treated as pilot-grade.

4.2 IETF protocol standards

Table 3b. Key IETF documents.

Document	Subject	Status (Oct 2026)
RFC 9794	PQ/T hybrid terminology	Informational RFC, June 2025 [26]
RFC 9954	Hybrid key exchange framework for TLS 1.3	Informational RFC, July 2026 [32]
RFC 10024	X25519MLKEM768, SecP256r1MLKEM768, SecP384r1MLKEM1024 for TLS 1.3	**Proposed Standard, Aug 2026** [16]. X25519MLKEM768 codepoint 0x11EC
draft-ietf-tls-mlkem	Pure ML-KEM key agreement for TLS 1.3	Internet-Draft
RFC 10042	mlkem768x25519-sha256, mlkem768nistp256-sha256, mlkem1024nistp384-sha384 for SSH	**RFC, 31 Aug 2026** [17]
RFC 9370	Multiple key exchanges in IKEv2	Proposed Standard, May 2023 [18]
RFC 9881	ML-DSA in X.509	Proposed Standard, Oct 2025 [19]
RFC 9909	SLH-DSA in X.509	Proposed Standard, Dec 2025 [20]
RFC 9935 / RFC 9936	ML-KEM in X.509 / in CMS	Proposed Standards, Mar 2026 [15], [22]
RFC 9629	KEMs in CMS (KEMRecipientInfo)	Proposed Standard, Aug 2024 [21]
draft-ietf-lamps-pq-composite-sigs	Composite ML-DSA (with RSA/ECDSA/EdDSA)	Internet-Draft (draft -13, Oct 2025); RFC publication not confirmed
RFC 8554 / RFC 8391	LMS / XMSS	Informational RFCs (2019 / 2018) [33], [34]
Merkle Tree Certificates	Compact PQ-friendly Web PKI authentication	Internet-Draft / experimental deployments; not standardized

RFC 10024 is notable because it formalized what browsers and CDNs had already deployed at scale from draft codepoints since 2024. It does *not* address authentication: TLS server certificates in the Web PKI remain ECDSA/RSA.

4.3 Other standards bodies

ETSI TS 103 744 specifies hybrid key-exchange constructions (including concatenation and cascade KDF combiners) for use with PQC [35]. At ISO/IEC, stateful hash-based signatures have been incorporated into ISO/IEC 14888-4, and ISO/IEC JTC 1/SC 27 is progressing work to add KEMs, including ML-KEM, FrodoKEM and Classic McEliece, to the ISO/IEC 18033-2 encryption standard. Assessors should treat ISO amendments as in progress unless a published edition is cited. Canada's ITSP.40.111 and Germany's TR-02102-1 provide national algorithm lists that may differ from NIST's; for example, BSI also accepts FrodoKEM and Classic McEliece [27].

4.4 Government guidance and timelines

United States.

- NSM-10 (4 May 2022) set the 2035 goal of mitigating quantum risk.
- The Quantum Computing Cybersecurity Preparedness Act (Public Law 117-260, 21 Dec 2022) required federal inventories and migration planning.
- OMB M-23-02 (18 Nov 2022) required annual cryptographic inventories of quantum-vulnerable systems [36].
- CISA, NSA and NIST jointly published "Quantum-Readiness: Migration to Post-Quantum Cryptography" (Aug 2023), urging inventories and vendor engagement [37].
- EO 14144 (16 Jan 2025) included PQC provisions. EO 14306 (6 Jun 2025) amended it, directing CISA to publish a list of product categories in which PQC-capable products are widely available, and requiring TLS 1.3 or a successor by 2 January 2030.
- CISA published "Product Categories for Technologies That Use Post-Quantum Cryptography Standards" on 23 January 2026 [38]. It lists cloud services (PaaS/IaaS), web software (browsers and servers), collaboration software, and endpoint security (data-at-rest and full-disk encryption) as categories where PQC is widely available. It names networking, identity and access management, operating systems, email, databases and others as still transitioning, and notes that most categories have implemented PQC for key establishment but not yet for signatures.
- EO 14412 (22 Jun 2026) set hard dates of 31 Dec 2030 (key establishment) and 31 Dec 2031 (signatures) for high-value assets and high-impact systems [9]. It directed a FAR proposed rule requiring covered contractors to meet PQC-incorporating FIPS by end-2030, directed acceleration of CMVP validations, and directed CISA and NIST to publish minimum CBOM elements within 270 days.
- OMB M-26-15 (24 Jun 2026) requires agency migration plans within 120 days (around 22 Oct 2026). It sets five phases: discovery 2026–27, pilots 2027–28, prioritized key-establishment migration 2028–30, signature migration 2031, and full migration by 2035 [10].

- CNSA 2.0 sets per-category dates [12]:
  - software/firmware signing: support and prefer by 2025, exclusive by 2030;
  - web browsers, servers and cloud: support and prefer by 2025, exclusive by 2033;
  - traditional networking equipment: support and prefer by 2026, exclusive by 2030;
  - operating systems: support and prefer by 2027, exclusive by 2033;
  - niche/constrained equipment and large PKI: support and prefer by 2030, exclusive by 2033;
  - custom and legacy applications: update or replace by 2033.

  The 2024 CNSSP 15 update requires new NSS acquisitions to be CNSA 2.0-compliant from 1 Jan 2027 and phase-out of non-supporting equipment by 31 Dec 2030.

Canada. The Cyber Centre's ITSM.40.001 roadmap (effective 23 June 2025) requires initial departmental migration plans by April 2026, completion for high-priority systems by end-2031, and completion for remaining systems by end-2035 [39]. groupitcs +6 Algorithm recommendations are in ITSP.40.111, and protocol configuration guidance in ITSP.40.062. cyber The Cyber Centre's "Preparing your organization for the quantum threat to cryptography" guidance and the Canadian Forum for Digital Infrastructure Resilience (CFDIR) Quantum-Readiness Best Practices address the private sector.

European Union. Commission Recommendation (EU) 2024/1101 (11 Apr 2024) led to the NIS Cooperation Group's Coordinated Implementation Roadmap (v1.1, June 2025) [40]. Its milestones are: national strategies, inventories and first steps by 31 Dec 2026; high-risk use cases and critical infrastructure migrated by 31 Dec 2030; and the transition completed for as many systems as feasible by 31 Dec 2035. An FAQ followed on 15 April 2026. The roadmap recommends standardized hybrid solutions. ENISA supports the effort, and the ECCG's Agreed Cryptographic Mechanisms v2 (May 2025) governs EUCC certification.

United Kingdom. NCSC's "Timelines for migration to post-quantum cryptography" (March 2025) sets discovery and a migration plan by 2028, highest-priority migrations 2028–2031, and completion by 2035 [41].

Germany, France and the Netherlands. BSI TR-02102-1 recommends PQC only in hybrid combination with classical schemes [27]. ANSSI's three-phase position foresees hybrid as defense in depth, then hybrid carrying the PQ security claim, and standalone PQC not before roughly 2030 [28]. The NIS Cooperation Group roadmap, whose PQC work stream is co-chaired by Germany, France and the Netherlands, states that "high-risk use cases should be transitioned to PQC as soon as possible, no later than by the end of 2030" [40]. The Dutch PQC Migration Handbook (AIVD, CWI and TNO; 1st ed. December 2023, revised 2024) provides a stepwise migration method organized around inventory, risk analysis and "no-regret" moves [42].

Australia. ASD's ISM does not approve traditional asymmetric cryptography (RSA, DH, ECDH, ECDSA) beyond the end of 2030. This is the most aggressive general-purpose cessation date of any Five Eyes member [43]. openssl-corporation

Table 3c. Comparative national timelines.

Jurisdiction / instrument	Binding?	Near-term milestone	High-priority deadline	Completion
US federal (EO 14412, OMB M-26-15)	Binding on agencies; FAR rule pending for contractors	Plans ~22 Oct 2026	PQ key establishment 31 Dec 2030; signatures 31 Dec 2031	2035
US NSS (CNSA 2.0, CNSSP 15)	Binding on NSS	New acquisitions compliant from 1 Jan 2027	Signing and networking exclusive 2030	2033 (most categories); NSS 2035
NIST IR 8547 (draft)	Proposal	—	Deprecate after 2030	Disallow after 2035
Canada (ITSM.40.001)	GC departments	Plans Apr 2026	High-priority end-2031	End-2035
EU (NIS CG roadmap)	Recommendation	Start by end-2026	High-risk/critical end-2030	End-2035 (as feasible)
UK (NCSC)	Guidance	Discovery and plan by 2028	Priority migrations 2028–2031	2035
Australia (ISM)	Binding on government	—	Traditional asymmetric not approved after end-2030	2030
Germany/France (BSI/ANSSI)	Guidance / certification	Hybrid now	Prioritized transitions by end-2030	—

5. Information Requirements for Assessment

This section catalogues what must be known about the software. Table 4 summarizes it; the subsections give rationale and detail.

5.1 A complete cryptographic inventory (CBOM)

Why it matters. You cannot assess what you cannot see. Every regime surveyed puts inventory first: M-23-02 and M-26-15, ITSM.40.001, the EU roadmap's 2026 milestone, NCSC's 2028 discovery milestone, and the Dutch handbook [10], [36], [39]–[42]. EO 14412 elevates the CBOM to a federal artifact with CISA/NIST-defined minimum elements due in 2027 [9].

What it must capture. For each cryptographic asset, the inventory should record:

- the algorithm, parameter set and key size;
- the mode or padding (for example AES-GCM, RSA-OAEP vs. PKCS#1 v1.5);
- the primitive *purpose*: key establishment, digital signature, encryption at rest, hashing, MAC, KDF, RNG/DRBG, or password hashing;
- the protocol context (TLS, SSH, IPsec, JOSE/COSE, CMS, proprietary);
- the implementing library and provider;
- the location in code or binary (file, function, call site);
- associated keys and certificates (type, size, validity, storage);
- the data classification served.

CycloneDX 1.6 (April 2024) introduced a native CBOM model with `cryptographic-asset` components (algorithm, certificate, protocol and related-crypto-material types) and dependency relationships, making the inventory machine-readable and diffable [44].

Acceptance criteria. The CBOM:

- covers first-party code, transitive dependencies, runtime configuration and platform services;
- is generated by tools and reconciled with runtime observation;
- is versioned with each release;
- records the purpose of every public-key use;
- carries no unexplained "unknown" entries.

5.2 Cryptographic libraries, providers, versions and the SBOM

Why it matters. Most applications inherit their cryptography from a small number of libraries, so PQ capability often reduces to *which version of which library with which configuration*. Examples as of 2026:

- **OpenSSL 3.5** (LTS, April 2025) natively provides ML-KEM, ML-DSA and SLH-DSA, with X25519MLKEM768 in default TLS key shares [45].
- **BoringSSL** and **AWS-LC** ship X25519MLKEM768; AWS-LC also exposes ML-DSA.
- The **Open Quantum Safe** project's liboqs and oqs-provider support experimentation with many algorithms but are explicitly not recommended for production without due diligence.
- **Bouncy Castle** (Java/C#) implements the FIPS PQC algorithms.
- **Go 1.24** added `crypto/mlkem`, with hybrid X25519MLKEM768 enabled by default in `crypto/tls`.
- **JDK 24** delivered ML-KEM (JEP 496) and ML-DSA (JEP 497) [46].
- **Microsoft** added ML-KEM and ML-DSA to Windows CNG/SymCrypt.
- **Mobile platforms** (Apple CryptoKit and Android/Conscrypt via BoringSSL) are adding ML-KEM and hybrid TLS support.

How to collect. Use SBOMs (CycloneDX/SPDX) from build systems, package-manager lock files, container image scans and binary fingerprinting of statically linked libraries.

Acceptance criteria. Each crypto library is identified to an exact version and build configuration. The version is one that implements the *final* FIPS algorithms (not pre-standard "Kyber round 3" variants, which are not interoperable with ML-KEM). The version is supported and patched, and the active provider (for example the OpenSSL FIPS provider vs. the default provider) is known.

5.3 Protocol configuration and negotiated behavior

Why it matters. Support is not use. A server built with OpenSSL 3.5 may still be configured with `Groups = X25519:P-256` and never negotiate a hybrid group. A client may offer X25519MLKEM768 to a server that ignores it.

Information needed:

- **TLS:** protocol versions (TLS 1.3 is required for the hybrid groups); supported and preferred named groups (`X25519MLKEM768` 0x11EC, `SecP256r1MLKEM768`, `SecP384r1MLKEM1024` per RFC 10024 [16]); key-share prediction behavior (to avoid HelloRetryRequest round trips); cipher suites (AES-256-GCM for CNSA 2.0 alignment); and signature algorithms.
- **SSH:** KEX algorithm lists and order (`mlkem768x25519-sha256`, standardized in RFC 10042 [17] and available since OpenSSH 9.9; `sntrup761x25519-sha512`); host-key and user-key algorithms, which are still classical in mainstream OpenSSH.
- **IPsec/IKEv2:** use of RFC 9370 additional key exchanges with ML-KEM, and the order of transforms [18].

- **Messaging:** Signal's PQXDH (announced September 2023) adds a Kyber-1024 KEM to the initial X3DH handshake [23]. Signal's Sparse Post-Quantum Ratchet (SPQR), announced 2 October 2025, is mixed with the Double Ratchet into a "Triple Ratchet" so that PQ protection also covers ongoing sessions [24]. Apple's iMessage PQ3 (February 2024) is a comparable design.
- **Application-layer formats:** JOSE/COSE (FN-DSA and ML-DSA drafts exist), CMS/S/MIME (RFC 9629/9936), and OpenPGP PQC drafts.

Acceptance criteria. Evidence from *observed* handshakes (packet captures or scanner output) shows a PQ or hybrid group selected for the relevant client population. Configuration files show PQ groups preferred. The share of connections negotiating PQ is measured, not assumed.

5.4 PKI and certificates

Information needed:

- signature algorithms at every level of each certificate chain (leaf, intermediates, root);
- the trust-anchor algorithms embedded in clients and devices;
- certificate lifetimes;
- CA and HSM capability to issue and protect ML-DSA/SLH-DSA keys (RFC 9881, RFC 9909) [19], [20];
- revocation infrastructure (CRL/OCSP signing algorithms);
- whether composite or dual-certificate strategies are planned.

Why it matters. Public Web PKI certificates are still classical. The CA/Browser Forum has not yet admitted ML-DSA roots, and the IETF is exploring Merkle Tree Certificates to avoid the bandwidth cost of placing several ML-DSA signatures in every handshake. Private PKIs, for example for device identity, code signing and service mesh, can move earlier.

Acceptance criteria.

- Private PKI roots and intermediates with lifetimes extending past about 2030 have a documented PQ (or composite) re-issuance plan, and the CA software and HSMs are demonstrably PQ-capable.
- Web-facing services record "authentication remains classical, pending ecosystem" as an explicit, accepted residual risk.

5.5 Code signing, firmware signing, secure boot and updates

Why it matters. Firmware roots of trust are often immutable for a device's lifetime, which may be 10–20 years. If the boot ROM verifies only ECDSA, the device can never be made PQ-secure. CNSA 2.0 sets 2030 as the exclusive-use date for software and firmware signing and specifies

LMS/XMSS (SP 800-208). The NSA's updated guidance also accepts ML-DSA once validated implementations are available [12], [29].

Information needed:

- the signature algorithm used by the bootloader, OS loader, update client and package manager;
- whether the verification key is in ROM, OTP fuses or updatable storage;
- the availability of a PQ-signed update path, and of dual signatures during transition;
- for LMS/XMSS: the state-management design (HSM-enforced counters, no state cloning or rollback, backup policy), because reusing a one-time key catastrophically breaks security;
- signing-service HSM support.

Acceptance criteria. New devices verify LMS/XMSS or ML-DSA signatures in their immutable root of trust, or can provably update the root of trust using a PQ-authenticated mechanism. Stateful-signature state handling follows SP 800-208's requirements, including generation and signing within validated hardware modules.

5.6 Symmetric and hash strength

Information needed: symmetric ciphers and key lengths, MAC and KDF algorithms, hash functions, and password-hashing parameters.

Acceptance criteria. Under NIST, AES-128+ and SHA-256+ are acceptable. Under CNSA 2.0, AES-256 and SHA-384/512 are required. Legacy algorithms (3DES, SHA-1, MD5) are absent, since they fail classically regardless.

5.7 Key management and hardware support

Information needed:

- key generation (randomness source, DRBG per SP 800-90A);
- key storage (HSMs, TPMs, cloud KMS, secure elements and smart cards) and whether these support ML-KEM/ML-DSA/LMS natively;
- the key-format choice for ML-KEM/ML-DSA (seed vs. expanded private key, which affects export and interoperability);
- rotation periods and procedures;
- key-wrapping algorithms;
- escrow and backup handling.

Why it matters. A PQ algorithm implemented in software but whose keys must be exported from the HSM in plaintext may be a net security regression. Many TPM 2.0 deployments do not yet support PQ algorithms, which constrains device attestation.

Acceptance criteria. PQ private keys are generated and used inside validated modules where policy requires it. The key-management system records algorithm metadata per key. Rotation to new algorithms has been exercised.

5.8 Data classification, confidentiality lifetime and system lifetime (Mosca inputs)

Information needed:

- data categories handled, with their required confidentiality periods (for example health records for decades, session tokens for minutes);
- signature validity requirements (legal and archival);
- system or device service life;
- estimated migration lead time, including vendor dependencies.

Why it matters. These inputs determine whether $x + y > z$ and therefore the priority and urgency of each finding [5]. Software handling only short-lived, low-value data may rationally accept classical authentication longer than a records system with 30-year confidentiality obligations.

Acceptance criteria. Each in-scope use case carries documented x and y values, and the risk prioritization explicitly uses them.

5.9 Implementation assurance

FIPS 140-3/CMVP and CAVP/ACVP. CAVP tests ML-KEM, ML-DSA and SLH-DSA through ACVP. CMVP module certificates show that the module containing them meets FIPS 140-3. The evidence to collect is the certificate numbers, the algorithm and parameter sets covered, the module version, and the operational environment. Note that validation is a bottleneck: EO 14412 explicitly orders NIST to accelerate CMVP processing [9]. "FIPS-validated" claims must therefore be checked against the public CMVP database for the *exact* module version and must include PQ algorithms in the approved-algorithm list.

Test vectors. Evidence of passing NIST ACVP vectors and adversarial suites such as Project Wycheproof (which includes ML-KEM and ML-DSA edge cases), plus the FIPS 203 mandatory key-validation checks.

Side channels and constant time. KyberSlash (2024) showed that several Kyber implementations leaked secret-dependent timing through variable-time division, enabling key recovery [47]. Evidence should include constant-time verification tooling (for example dudect- or valgrind-taint-style analyses, or compiler-level checks), a review of secret-dependent branches and memory access, and, for embedded targets, power and EM analysis and fault-injection resistance appropriate to the threat model. Fault attacks on deterministic signature schemes are a known concern: ML-DSA's "hedged" signing mode is recommended over the fully deterministic variant.

Formal verification. Several ML-KEM implementations now carry machine-checked proofs:

- Cryspen's **libcrux** (Rust), which is "formally verified for panic freedom, correctness, and secret independence in F* using the hax toolchain";
- the PQCA **mlkem-native** (the default ML-KEM in liboqs and AWS-LC), whose AArch64 and x86_64 assembly "is proved to be functionally correct, memory-safe, and of secret-independent timing (constant-time), using HOL-Light", with its C code proved memory- and type-safe via CBMC;
- the Formosa project's **libjade**, Jasmin implementations proven in EasyCrypt [48].

These reduce, but do not eliminate, implementation risk, because proofs cover specific properties and platforms.

Known vulnerabilities. Review the CVE history of the specific library versions, including PQ-specific advisories.

Acceptance criteria. The PQ implementation is CAVP-certified at minimum, and inside a FIPS 140-3 validated module where policy requires. It has documented constant-time evidence and no unpatched relevant CVEs. Preferably it has formally verified cores.

5.10 Hybrid design choices and combiner correctness

Information needed:

- whether the software uses hybrid or pure PQ, and the justification by reference to its regulatory regime (CNSA 2.0 vs. BSI/ANSSI) [12], [27], [28];
- the exact combiner (RFC 10024's concatenation into the TLS 1.3 key schedule, RFC 10042's hashing in SSH, ETSI TS 103 744 KDFs, X-Wing, or composite signatures);
- for FIPS environments, which component is the approved one. RFC 10024 notes that for X25519MLKEM768 the ML-KEM part must be certified, whereas for the NIST-curve hybrids the ECDHE part may carry certification [16].

Acceptance criteria. The combiner is drawn from a standard or an analyzed construction. No hand-rolled XOR or truncation of secrets is used. The rationale is documented.

5.11 Crypto-agility

Information needed:

- whether algorithms are negotiated or hard-coded;
- whether algorithm choices sit behind an abstraction layer or provider interface;
- whether configuration changes can switch algorithms without recompilation;
- whether protocol formats carry algorithm identifiers and can accommodate larger keys;

- whether keys and certificates can be rotated fleet-wide, and how long an algorithm swap took in practice.

NIST CSWP 39 frames these capabilities [25].

Acceptance criteria. An algorithm change has been demonstrated in test, through policy and configuration rather than code changes, within a defined time objective. Hard-coded algorithm OIDs and fixed-size buffers are absent.

5.12 Downgrade, fallback and interoperability

Information needed:

- behavior when the peer does not support PQ (silent fallback to classical, warning, or refusal);
- protection against an active attacker stripping PQ options (TLS 1.3 transcript binding protects the negotiated group, but application-level retry-on-failure logic can reintroduce downgrades);
- interoperability test results with major clients and servers;
- middlebox tolerance of larger ClientHello messages.

Acceptance criteria.

- Policies define where classical fallback is permitted, and where it is not (for example internal service-to-service links).
- Downgrade tests pass.
- Interoperability is documented against named peer implementations.

5.13 Performance and size constraints

Information needed:

- handshake size increases (per RFC 10024, an X25519MLKEM768 client key share is 1,216 bytes, of which 1,184 are ML-KEM, and the server share is 1,120 bytes, of which 1,088 are ML-KEM [16]);
- certificate-chain growth if ML-DSA is used (several kilobytes per signature and key; see Table 2);
- latency;
- MTU and fragmentation effects, especially for DTLS, IKEv2 over UDP and QUIC initial packets;
- CPU, RAM, flash and stack budgets on constrained devices;
- signature-verification time in boot paths.

Acceptance criteria. Measured overhead is within service-level objectives on representative hardware and networks, and constrained devices have headroom for the chosen parameter sets.

5.14 Vendor and supplier information

Information needed:

- vendor PQC roadmaps with dated milestones;
- a CBOM or cryptographic attestation for the product;
- lists of supported algorithms and protocols, with defaults;
- validation certificates;
- update and agility commitments;
- end-of-support dates relative to 2030/2035;
- contractual clauses.

The Canadian roadmap expressly calls for PQC requirements in procurement [39]. CISA's product-category list tells US agencies to procure only PQC-capable products in the widely available categories [38]. M-26-15 requires a third-party coordination plan [10].

Acceptance criteria. Signed attestations reference specific product versions and standards (FIPS 203/204/205, RFC 10024 and similar). They are corroborated by independent testing where feasible. Contracts contain PQ-migration and CBOM-delivery clauses.

5.15 Compliance mapping

Information needed: applicable regimes, by jurisdiction and sector (EO 14412/M-26-15, CNSA 2.0/CNSSP 15, FedRAMP, ITSM.40.001, the EU roadmap and NIS2/DORA, NCSC, ISM), and the deadlines that apply to each use case.

Acceptance criteria. Every finding is traced to the requirement and date it affects.

Table 4. Master table of information requirements.

#	Information item	Why it matters	How to collect	Acceptance criteria ("good")
1	Cryptographic inventory / CBOM	Basis of every other judgment; mandated first step in US, CA, EU, UK	Static and binary analysis, SBOM enrichment, runtime discovery; CycloneDX 1.6+ CBOM	Complete, versioned, purpose-tagged; no unexplained unknowns; reconciled with runtime
2	Crypto libraries, providers, versions (SBOM)	PQ capability is largely inherited	SBOM, lock files, image scans, binary fingerprinting	Versions implement final FIPS 203/204/205; supported; active provider known
3	TLS/SSH/IPsec/messaging configuration and negotiated groups	Support ≠ use; HNDL exposure is determined by negotiation	Config review; testssl.sh/sslyze/nmap; packet capture of key_share	PQ/hybrid group (e.g., X25519MLKEM768) observed as negotiated; share of PQ connections measured
4	PKI chains, trust anchors, CA/HSM readiness	Long-lived trust anchors face forge-later risk	Certificate scans, CA configuration, HSM specifications	Private PKI plan for ML-DSA/SLH-DSA (RFC 9881/9909) or composite; documented residual for Web PKI
5	Code/firmware signing, secure boot, update path	Immutable roots of trust; CNSA 2.0 2030 deadline	Boot-chain documentation, firmware RE, signing-service review	LMS/XMSS or ML-DSA verification in root of trust or PQ-authenticated root update; SP 800-208 state management
6	Symmetric/hash strength	Grover margin; CNSA 2.0 requires AES-256/SHA-384	CBOM, configuration	AES-128+/SHA-256+ (NIST) or AES-256/SHA-384+ (CNSA 2.0); no legacy primitives

#	Information item	Why it matters	How to collect	Acceptance criteria ("good")
7	Key management and hardware support	Insecure key handling undermines PQ algorithms	KMS/HSM/TPM inventories and specifications	PQ keys generated and used in validated modules; rotation exercised
8	Data lifetime, system/device lifetime, migration time	Mosca $x + y > z$ drives priority	Data classification, asset registers, interviews	x, y documented per use case; prioritization uses them
9	CMVP/CAVP certificates	Independent conformance and module assurance	NIST CMVP/CAVP databases	Certificates match exact version and include ML-KEM/ML-DSA/SLH-DSA
10	Test vectors (ACVP, Wycheproof)	Correctness and edge-case handling	CI logs, test reports	All vectors pass; FIPS 203 input checks enforced
11	Side-channel/fault evidence	KyberSlash-class leaks break PQ in practice	Constant-time tooling, lab reports	Documented constant-time analysis; physical-attack testing for exposed devices
12	Formal verification	Reduces implementation risk	Vendor/library documentation	Verified cores (libcrux, mlkem-native, libjade) preferred
13	CVE history	Known flaws	NVD, vendor advisories	No unpatched relevant CVEs
14	Hybrid vs. pure choice and combiner	Hedges new-algorithm risk; regime-dependent	Design documentation, protocol traces	Standardized combiner; rationale mapped to regime
15	Crypto-agility	Future algorithm changes (FN-	Architecture review, change drills (CSWP 39)	Algorithm swap demonstrated via

#	Information item	Why it matters	How to collect	Acceptance criteria ("good")
		DSA, HQC, breaks)		configuration within a time objective
16	Downgrade/fallback, interoperability	Attackers or misconfiguration can strip PQ	Active downgrade tests, interop matrices	Fallback policy defined; downgrade tests pass
17	Performance/size constraints	Deployability; MTU and constrained devices	Benchmarks, network tests	Within SLOs; no fragmentation failures
18	Vendor roadmaps, attestations, contracts	Third-party code dominates estates	Questionnaires, attestations, contracts	Dated, version-specific, corroborated commitments; CBOM delivery
19	Compliance mapping	Determines deadlines and required posture	Legal and regulatory review	Each finding mapped to requirement and date

6. Evidence-Collection Methods and Tools

No single method finds all cryptography. A defensible assessment *triangulates* static, binary, dependency, runtime and documentary evidence. Table 5 summarizes the methods and their characteristic false negatives.

6.1 Static source analysis

Static source analysis finds API calls to cryptographic libraries and hard-coded algorithm choices. Tools include:

- the PQCA's **Sonar Cryptography Plugin** and **CBOMkit**, which generate CycloneDX CBOMs from Java and Python code;
- **CodeQL** and **Semgrep** crypto rule packs;
- **CogniCrypt/CryptoAnalysis**, which checks Java Cryptography Architecture usage against CrySL specifications;
- **CryptoGuard**, which detects crypto misuse in Java and Android.

Limitations: these tools are language-specific and miss cryptography reached through reflection, configuration strings, native code, or algorithms selected at runtime by negotiation. They also miss custom implementations without recognizable APIs.

6.2 Binary analysis

Binary analysis identifies cryptographic constants (S-boxes, NTT twiddle factors, curve parameters), library signatures and symbol tables in compiled executables and firmware, using tools such as signsrch-style constant scanners and FLIRT signatures in IDA/Ghidra. It is essential for vendor software and firmware.

Limitations: obfuscation, stripping, inlining and compiler optimization defeat signatures. Detecting ML-KEM constants does not show that ML-KEM is *used*.

6.3 Dependency and SBOM analysis

Dependency and SBOM analysis maps libraries and versions, using tools such as Syft, Trivy, OWASP Dependency-Track and CycloneDX generators.

Limitations: an SBOM shows presence, not configuration or use. Statically linked or vendored code is often missed, and OS-provided libraries may be outside the scan.

6.4 Runtime and network discovery

- **testssl.sh** and **sslyze** enumerate TLS versions, cipher suites and supported groups. Recent versions test for ML-KEM hybrids.
- **nmap** scripts (ssl-enum-ciphers, ssh2-enum-algos) list SSH KEX algorithms, including `mlkem768x25519-sha256`.
- **Packet capture** with Wireshark inspects the `supported_groups` and `key_share` extensions and identifies the server's selected group, the definitive evidence of negotiation.
- eBPF or library hooks observe live calls.

Limitations: these methods see only exercised paths and reachable endpoints. Internal east-west traffic, rarely used features and data-at-rest cryptography are invisible to them. The results also depend on the client used for probing.

6.5 Commercial discovery and inventory platforms

Commercial platforms include IBM Quantum Safe Explorer (source and binary scanning to CBOM) and IBM Guardium Quantum Safe, along with products from Keyfactor, SandboxAQ (AQtive Guard), InfoSec Global/Keyfactor AgileSec, QuSecure and Venafi/CyberArk. These combine host agents, network sensors and certificate discovery. NCCoE SP 1800-38B documents functional testing of several such tools [31].

Limitations: coverage claims are vendor-reported, outputs are often not interoperable, and agents may not run on embedded targets.

6.6 Configuration review, interviews and questionnaires

Configuration review, interviews and questionnaires capture design intent, data lifetimes, roadmaps and process evidence (key ceremonies, LMS state handling) that no scanner can see.

Limitations: they are subject to respondent knowledge and optimism bias, so claims require corroboration.

Table 5. Methods versus evidence types.

Method	Best for	Typical false negatives
Static source analysis	First-party code, CBOM generation	Native code, reflection, configuration-driven algorithms, custom crypto
Binary analysis	Vendor software, firmware	Obfuscated/inlined code; presence ≠ use
SBOM/dependency	Library versions	Static linking, OS libraries, configuration
Runtime/network	Negotiated reality	Unexercised paths, internal traffic, data at rest
Commercial platforms	Scale, continuous inventory	Unverified coverage, embedded blind spots
Interviews/questionnaires	Intent, lifetimes, processes	Unverified or optimistic claims

7. Proposed Assessment Framework

The framework synthesizes several sources: the NCCoE migration practice guide [31], the CISA/NSA/NIST quantum-readiness factsheet [37], the Canadian roadmap's inventory-plan-migrate phases [39], the Dutch handbook's inventory and risk-analysis steps [42], M-26-15's phase structure [10], and NIST CSWP 39's agility model [25]. It also reflects the academic literature on crypto-agility risk assessment and quantum risk scoring, which extends Mosca's binary inequality into multi-factor scores.

7.1 Stages

1. **Scoping.** Define the software boundary, deployment contexts (web/cloud, desktop, mobile, library/SDK, embedded), applicable regimes and deadlines, and assessment

mode (white-box, grey-box or black-box). Output: a scope statement and a requirements baseline (NIST general-purpose vs. CNSA 2.0; hybrid-required vs. pure-PQ-acceptable).

2. **Discovery and inventory.** Run the triangulated methods of Section 6 and produce a CBOM. Output: CBOM v1 with coverage notes and known blind spots.
3. **Classification and risk prioritization.** Tag each asset by purpose (key establishment / signature / symmetric), quantum vulnerability, data lifetime *x*, migration time *y* and exposure (internet-facing, HNDL-capturable). Rank by Mosca exposure and regulatory deadline. Output: a prioritized risk register.
4. **Conformance assessment.** Compare each asset against the algorithm and protocol requirements of Sections 3–4, using items 1–6 and 14 of Table 4. Output: conformance findings per use case.
5. **Implementation assurance.** Evaluate items 9–13 of Table 4 (validation, vectors, side channels, formal proofs, CVEs), plus agility and downgrade testing (items 15–16). Output: assurance findings.
6. **Verdict and reporting.** Assign a readiness level (Section 7.3) per use case and overall, with residual risks, deadlines and remediation. Re-run on each release or when standards change.

7.2 Decision matrix

Table 6. Per-use-case decision matrix.

Question	If "No"	If "Yes" →
Q1. Is the use case's cryptography fully inventoried?	Verdict: *Indeterminate*	Q2
Q2. Does the use case rely on quantum-vulnerable public-key crypto for confidentiality (key establishment)?	Q4	Q3
Q3. Is a final-standard PQ or hybrid KEM negotiated in practice (observed)?	*Quantum-vulnerable (HNDL-exposed)*; priority per Mosca	Q4
Q4. Does it rely on quantum-vulnerable signatures for authentication or integrity?	Q6	Q5
Q5. Are signatures PQ (ML-DSA/SLH-DSA/LMS/XMSS) or composite throughout the chain/root of trust?	*Hybrid-KEX only*; record forge-later residual and deadline (2031 US federal; 2030 CNSA 2.0 firmware)	Q6
Q6. Is the implementation assured (CAVP/CMVP as required, constant-time evidence, no open CVEs)?	Downgrade by one level; remediation required	Q7
Q7. Are downgrade resistance and agility demonstrated?	Cap at level 3	Level 4

7.3 Readiness and maturity scale

- **Level 0, Unknown:** no adequate inventory.
- **Level 1, Quantum-vulnerable:** classical-only key establishment and signatures.
- **Level 2, PQ-capable:** libraries and products support PQ, but it is not enabled or negotiated in practice.
- **Level 3, Hybrid PQ key establishment:** hybrid or pure PQ KEM negotiated for in-scope traffic, with authentication still classical. This is HNDL-mitigated.
- **Level 4, PQ-secure:** PQ or hybrid key establishment *and* PQ/composite authentication, including roots of trust, with implementation assurance.
- **Level 5, Fully PQ-secure and crypto-agile:** Level 4, plus demonstrated algorithm agility, downgrade resistance and continuous CBOM-driven monitoring.

Most internet-facing software in October 2026 can reach Level 3. Level 4 for Web PKI authentication is not yet achievable for reasons outside the software owner's control. Private and embedded systems can reach Level 4 now.

---

8. Illustrative Case Study

8.1 System

"ExampleCorp" operates a customer-facing REST API behind a load balancer (nginx linked against OpenSSL), with an administrative OpenSSH estate of about 400 Linux hosts. It also ships IoT gateways with a 15-year service life that receive signed firmware updates. Data handled includes financial records with a 10-year confidentiality requirement. ExampleCorp is a US federal contractor, so the forthcoming FAR rule under EO 14412 is relevant.

8.2 Evidence gathered

Scoping.

- Regime: EO 14412 contractor obligations (expected end-2030), plus customer contractual requirements.
- Mosca inputs: x = 10 years for API data. y is estimated at 3 years for the API and SSH estate, and 6 years for gateways, because of the hardware root of trust.

Discovery (CBOM).

- The SBOM shows nginx linked to OpenSSL 3.0.13 on half the fleet and OpenSSL 3.5.x on the other half.
- Static analysis of the API code shows JWT signing with ES256 (ECDSA P-256) and AES-256-GCM for field-level encryption.
- Binary analysis of gateway firmware finds the ECDSA P-256 verification routine in the boot ROM and a software update client that verifies ECDSA-signed images.

Protocol evidence.

- testssl.sh and packet captures from a current Chrome client show `X25519MLKEM768` (0x11EC) negotiated on the 3.5 nodes and X25519 on the 3.0 nodes. About 50% of handshakes are PQ-protected.
- The server certificate chain is ECDSA P-384, issued by a public CA.
- nmap `ssh2-enum-algos` shows that OpenSSH 9.6 hosts offer `sntrup761x25519-sha512`, while hosts updated to OpenSSH 10.x offer and prefer `mlkem768x25519-sha256` (RFC 10042). Host keys are Ed25519.

Implementation assurance.

- OpenSSL 3.5's FIPS provider status for ML-KEM is checked against the CMVP database. The module certificate in use covers only classical algorithms, so ML-KEM is running from the default provider, which is not validated.
- No relevant open CVEs are found.
- The gateway vendor's questionnaire states "PQC roadmap 2028" with no algorithm specified.

Agility.

- TLS groups are configuration-driven (Level-5 capable).
- JWT algorithms are hard-coded in two microservices.
- The gateway root of trust is immutable.

8.3 Verdicts

Use case	Evidence summary	Level	Priority / action
Public API TLS key establishment	Hybrid negotiated on 50% of nodes	2→3 (mixed)	**High** (HNDL, $x+y = 13 >$ plausible z): upgrade all nodes to OpenSSL 3.5; set `Groups X25519MLKEM768:X25519`; obtain a validated module covering ML-KEM before the FAR deadline
Public API authentication (X.509, JWT)	ECDSA throughout	1	Medium: Web PKI dependent; refactor JWT to make algorithms configurable; pilot ML-DSA tokens internally; target 2031
SSH administration	Mixed sntrup/ML-KEM hybrid; Ed25519 host keys	3 (after upgrade)	Upgrade to OpenSSH ≥ 9.9 fleet-wide and prefer `mlkem768x25519-sha256`; monitor for PQ host-key standards
Gateway firmware updates	Immutable ECDSA root of trust	1 (cannot exceed 1 for installed base)	**Critical** for new hardware: require LMS/XMSS or ML-DSA in the root of trust (CNSA 2.0-aligned 2030); for the installed base, document residual risk and replacement schedule

Overall verdict. *Not PQ-secure.* The software is at Level 2–3 for confidentiality and Level 1 for authentication. The highest-risk exposures are HNDL against API data on non-upgraded nodes and the immutable firmware root of trust. The case illustrates why the verdict must be scoped per use case. It also shows that "supports ML-KEM" (true for the codebase) diverges

from “negotiates ML-KEM” (true for only half the traffic) and from “validated ML-KEM” (not yet true).

---

9. Discussion: Challenges, Limitations and Open Problems

Cryptanalytic risk of newer algorithms. In 2022, SIKE, a NIST fourth-round KEM candidate, was broken classically by Castryck and Decru’s key-recovery attack. The preliminary version (IACR ePrint 2022/975) reports that it “breaks the instantiation SIKEp434... in about one hour on a single core”, and the EUROCRYPT 2023 version reduces this to about ten minutes. Rainbow, a third-round signature finalist, was broken by Beullens, whose attack “returns the corresponding secret key after on average 53 hours (one weekend) of computation time on a standard laptop” (CRYPTO 2022) [49], [50]. These breaks underpin the European preference for hybrids and NIST’s pursuit of algorithmic diversity through HQC and the additional-signature round [13], [14]. ML-KEM and ML-DSA have withstood extensive analysis, but a PQ-secure verdict is necessarily conditional on the state of cryptanalysis.

Implementation immaturity. KyberSlash [47] and similar findings show that implementation, rather than mathematics, is the near-term weak point. Side-channel work on ML-DSA (rejection sampling, signing) and fault attacks on hash-based and lattice signatures continue to appear. Formal verification is advancing but covers only some libraries and properties.

PQ PKI immaturity. RFC 9881 and RFC 9909 define ML-DSA and SLH-DSA certificates, but browsers, CAs and the CA/Browser Forum have not deployed PQ Web PKI roots. Composite-signature standardization is unfinished, and size-reducing designs such as Merkle Tree Certificates are experimental. Authentication is therefore the long pole for internet-facing software.

Constrained and legacy systems. Devices with immutable ECDSA roots of trust, small MTUs, or limited RAM and flash may be impossible to migrate in place. CNSA 2.0’s “update or replace by 2033” language reflects that replacement is sometimes the only remedy [12].

No formal “PQ-secure” certification. FIPS 140-3 validates modules and algorithms, and Common Criteria and EUCC evaluate products against profiles. No scheme certifies that *software as deployed* is PQ-secure. CISA’s product-category list [38] is a procurement aid, not a certification. Verdicts thus remain assessor judgments, which is why the evidence trail matters.

Standards churn. FIPS 206 and the HQC standard are pending, IR 8547 is a draft, and composite signatures are unfinished. Any verdict must be dated and revisited, which makes agility evidence (Table 4, item 15) essential.

Timeline uncertainty. Resource estimates fell by roughly two orders of magnitude between 2019 and 2026 [2]–[4]. postquantum postquantum Expert surveys yield probability ranges, not dates [5]. Mosca's inequality turns this uncertainty into decisions, but the value of z remains contested.

QKD is not a substitute. The NSA and the UK NCSC have both stated that quantum key distribution is not a substitute for PQC. QKD provides only partial solutions (key distribution without source authentication), requires special-purpose hardware and links, has implementation-dependent security that is difficult to validate, raises insider and denial-of-service risks, and still needs classical or PQ authentication [51], [52]. PQC is therefore the primary migration path. QKD evidence should not be accepted in place of PQ algorithm evidence.

Limitations of this paper. Several status claims rely partly on secondary reporting cross-checked against primary documents, notably the FIPS 206 drafting status, the HQC "FIPS 207" designation and the contents of CISA's category tables. Library feature claims reflect public release notes at the time of writing and change quickly. The case study is illustrative, not empirical.

---

10. Conclusion and Recommendations

Determining whether software is post-quantum secure is an evidence problem, not a feature checkbox. The information required spans:

- what cryptography exists (the CBOM and SBOM);
- what is *negotiated* (protocol evidence);
- how well it is built (validation, test vectors, side-channel evidence, formal proofs, CVEs);
- how it is operated (key management, PKI, firmware roots of trust, downgrade policy, agility);
- why it matters (data and system lifetimes, regulatory deadlines).

As of October 2026, the standards make PQ *key establishment* achievable and expected now, through FIPS 203, RFC 10024, RFC 10042 and RFC 9370. PQ *authentication* is standardized at the algorithm and certificate-format level but remains constrained by PKI ecosystems.

Recommendations:

1. **Inventory first and continuously.** Produce a CycloneDX CBOM per release, triangulated across static, binary, dependency and runtime evidence, and prepare to conform to the CISA/NIST minimum CBOM elements required by EO 14412.
2. **Prioritize HNDL exposure.** Enable hybrid ML-KEM (X25519MLKEM768, or SecP384r1MLKEM1024 where Category 5 or FIPS-curve constraints apply) on all TLS 1.3

endpoints. Enable `mlkem768x25519-sha256` for SSH, and RFC 9370 ML-KEM for IKEv2. Verify negotiation by capture, not configuration alone.

3. **Fix roots of trust now.** Require LMS/XMSS or ML-DSA verification in the boot and update chains of any device shipping today with a service life past 2030.
4. **Demand assurance, not claims.** Require CAVP/CMVP certificates that cover PQ algorithms for the exact module version, constant-time evidence, and CVE transparency. Prefer formally verified cores.
5. **Choose hybrid versus pure deliberately.** Use hybrids where European or general-purpose guidance applies. Plan for pure Category 5 PQ where CNSA 2.0 applies. Accept only standardized combiners.
6. **Engineer and test agility.** Follow NIST CSWP 39. Make algorithms configuration-driven, and rehearse an algorithm swap, since FN-DSA, HQC or a cryptanalytic surprise will require one.
7. **Contract for it.** Embed PQ roadmaps, CBOM delivery and validation commitments in procurement, aligned with CISA's product categories and the FAR rule expected under EO 14412.
8. **Report graded, dated verdicts.** Use the five-level scale per use case, record residual risks (especially Web PKI authentication) explicitly, and reassess when FIPS 206, the HQC standard or the final IR 8547 is published.

References

[1] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM J. Comput.*, vol. 26, no. 5, pp. 1484–1509, 1997. doi:10.1137/S0097539795293172.

[2] C. Gidney and M. Ekerå, "How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits," *Quantum*, vol. 5, p. 433, 2021. doi:10.22331/q-2021-04-15-433. arXiv:1905.09749.

[3] C. Gidney, "How to factor 2048 bit RSA integers with less than a million noisy qubits," arXiv:2505.15917, May 2025. doi:10.48550/arXiv.2505.15917.

[4] P. Webster, L. Berent, O. Chandra, E. T. Hockings, N. Baspin, F. Thomsen, S. C. Smith, and L. Z. Cohen, "The Pinnacle Architecture: Reducing the cost of breaking RSA-2048 to 100 000 physical qubits using quantum LDPC codes," arXiv:2602.11457, Feb. 2026. doi:10.48550/arXiv.2602.11457.

[5] M. Mosca and M. Piani, *Quantum Threat Timeline Report 2025*, Global Risk Institute / evolutionQ, Mar. 2026. https://globalriskinstitute.org/publication/quantum-threat-timeline-report-2025b/

[6] NIST, *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard*, Aug. 2024. doi:10.6028/NIST.FIPS.203.

[7] NIST, *FIPS 204: Module-Lattice-Based Digital Signature Standard*, Aug. 2024. doi:10.6028/NIST.FIPS.204.

[8] NIST, *FIPS 205: Stateless Hash-Based Digital Signature Standard*, Aug. 2024. doi:10.6028/NIST.FIPS.205.

[9] Executive Order 14412, "Securing the Nation Against Advanced Cryptographic Attacks," 22 Jun. 2026, *Federal Register*, 25 Jun. 2026. https://www.federalregister.gov/documents/2026/06/25/2026-12909/securing-the-nation-against-advanced-cryptographic-attacks

[10] Office of Management and Budget, M-26-15, "Execution of the Migration to Post-Quantum Cryptography," 24 Jun. 2026. https://www.whitehouse.gov/wp-content/uploads/2026/06/M-26-15-Execution-of-the-Migration-to-Post-Quantum-Cryptography.pdf

[11] D. Moody et al., *NIST IR 8547 (Initial Public Draft): Transition to Post-Quantum Cryptography Standards*, NIST, Nov. 2024. https://nvlpubs.nist.gov/nistpubs/ir/2024/NIST.IR.8547.ipd.pdf

[12] National Security Agency, "Announcing the Commercial National Security Algorithm Suite 2.0," Cybersecurity Advisory, Sep. 2022, updated May 2025; and "The Commercial National Security Algorithm Suite 2.0 and Quantum Computing FAQ." https://media.defense.gov/2025/May/30/2003728741/-1/-1/0/CSA_CNSA_2.0_ALGORITHMS.PDF

[13] NIST, *NIST IR 8545: Status Report on the Fourth Round of the NIST Post-Quantum Cryptography Standardization Process*, Mar. 2025. https://nvlpubs.nist.gov/nistpubs/ir/2025/NIST.IR.8545.pdf

[14] NIST, *NIST IR 8610: Status Report on the Second Round of the Additional Digital Signature Schemes for the NIST Post-Quantum Cryptography Standardization Process*, May 2026. https://csrc.nist.gov/pubs/ir/8610/final

[15] S. Turner, P. Kampanakis, J. Massimo, and B. E. Westerbaan, "Internet X.509 Public Key Infrastructure – Algorithm Identifiers for the Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM)," RFC 9935, Mar. 2026. doi:10.17487/RFC9935.

[16] K. Kwiatkowski, P. Kampanakis, B. E. Westerbaan, and D. Stebila, "Post-Quantum Traditional (PQ/T) Hybrid Key Agreement Mechanisms for TLS 1.3," RFC 10024, Aug. 2026. https://www.rfc-editor.org/info/rfc10024

[17] P. Kampanakis, D. Stebila, and T. Hansen, “Post-Quantum/Traditional Hybrid Key Exchange with the Module-Lattice-Based Key-Encapsulation Mechanism for Use in SSH,” RFC 10042, Aug. 2026. doi:10.17487/RFC10042.

[18] C. Tjhai et al., “Multiple Key Exchanges in the Internet Key Exchange Protocol Version 2 (IKEv2),” RFC 9370, May 2023. doi:10.17487/RFC9370.

[19] J. Massimo, P. Kampanakis, S. Turner, and B. E. Westerbaan, “Internet X.509 Public Key Infrastructure – Algorithm Identifiers for the Module-Lattice-Based Digital Signature Algorithm (ML-DSA),” RFC 9881, Oct. 2025. doi:10.17487/RFC9881.

[20] K. Bashiri, S. Fluhrer, S.-L. Gazdag, D. Van Geest, and S. Kousidis, “Internet X.509 Public Key Infrastructure – Algorithm Identifiers for the Stateless Hash-Based Digital Signature Algorithm (SLH-DSA),” RFC 9909, Dec. 2025. doi:10.17487/RFC9909.

[21] R. Housley, J. Gray, and T. Okubo, “Using Key Encapsulation Mechanism (KEM) Algorithms in the Cryptographic Message Syntax (CMS),” RFC 9629, Aug. 2024. doi:10.17487/RFC9629.

[22] J. Prat, M. Ounsworth, and D. Van Geest, “Use of ML-KEM in the Cryptographic Message Syntax (CMS),” RFC 9936, Mar. 2026. doi:10.17487/RFC9936.

[23] E. Kret and R. Schmidt, “Quantum Resistance and the Signal Protocol,” Signal blog, Sep. 2023. https://signal.org/blog/pqxdh/

[24] G. Connell and R. Schmidt, “Signal Protocol and Post-Quantum Ratchets,” Signal blog, 2 Oct. 2025. https://signal.org/blog/spqr/

[25] NIST, *CSWP 39: Considerations for Achieving Crypto Agility: Strategies and Practices*, Dec. 2025, updated 29 Jun. 2026 (CSWP 39upd1). doi:10.6028/NIST.CSWP.39-upd1.

[26] F. Driscoll, M. Parsons, and B. Hale, “Terminology for Post-Quantum Traditional Hybrid Schemes,” RFC 9794, Jun. 2025. doi:10.17487/RFC9794.

[27] Bundesamt für Sicherheit in der Informationstechnik, *Technical Guideline TR-02102-1: Cryptographic Mechanisms: Recommendations and Key Lengths*, version 2025-1. https://www.bsi.bund.de/

[28] ANSSI, “ANSSI views on the Post-Quantum Cryptography transition,” 2022, with 2023 follow-up. https://cyber.gouv.fr/en/publications/anssi-views-post-quantum-cryptography-transition

[29] D. Cooper et al., *NIST SP 800-208: Recommendation for Stateful Hash-Based Signature Schemes*, Oct. 2020. doi:10.6028/NIST.SP.800-208.

[30] G. Alagic et al., *NIST SP 800-227: Recommendations for Key-Encapsulation Mechanisms*, Sep. 2025. doi:10.6028/NIST.SP.800-227.

[31] NIST NCCoE, *SP 1800-38A/B/C: Migration to Post-Quantum Cryptography* (preliminary drafts). https://www.nccoe.nist.gov/applied-cryptography/migration-to-pqc

[32] D. Stebila, S. Fluhrer, and S. Gueron, "Hybrid Key Exchange in TLS 1.3," RFC 9954, Jul. 2026. https://www.rfc-editor.org/info/rfc9954

[33] D. McGrew, M. Curcio, and S. Fluhrer, "Leighton-Micali Hash-Based Signatures," RFC 8554, Apr. 2019. doi:10.17487/RFC8554.

[34] A. Huelsing, D. Butin, S. Gazdag, J. Rijneveld, and A. Mohaisen, "XMSS: eXtended Merkle Signature Scheme," RFC 8391, May 2018. doi:10.17487/RFC8391.

[35] ETSI, *TS 103 744: CYBER; Quantum-safe Hybrid Key Exchanges*. https://www.etsi.org/

[36] Office of Management and Budget, M-23-02, "Migrating to Post-Quantum Cryptography," 18 Nov. 2022. https://www.whitehouse.gov/wp-content/uploads/2022/11/M-23-02-M-Memo-on-Migrating-to-Post-Quantum-Cryptography.pdf

[37] CISA, NSA, and NIST, "Quantum-Readiness: Migration to Post-Quantum Cryptography," Aug. 2023. https://www.cisa.gov/resources-tools/resources/quantum-readiness-migration-post-quantum-cryptography

[38] CISA, "Product Categories for Technologies That Use Post-Quantum Cryptography Standards," 23 Jan. 2026. https://www.cisa.gov/resources-tools/resources/product-categories-technologies-use-post-quantum-cryptography-standards

[39] Canadian Centre for Cyber Security, *Roadmap for the migration to post-quantum cryptography for the Government of Canada (ITSM.40.001)*, Jun. 2025. https://www.cyber.gc.ca/en/guidance/roadmap-migration-post-quantum-cryptography-government-canada-itsm40001

[40] NIS Cooperation Group, *A Coordinated Implementation Roadmap for the Transition to Post-Quantum Cryptography*, v1.1, Jun. 2025. https://digital-strategy.ec.europa.eu/en/library/coordinated-implementation-roadmap-transition-post-quantum-cryptography

[41] UK National Cyber Security Centre, "Timelines for migration to post-quantum cryptography," Mar. 2025. https://www.ncsc.gov.uk/guidance/pqc-migration-timelines

[42] AIVD, CWI, and TNO, *The PQC Migration Handbook: Guidelines for Migrating to Post-Quantum Cryptography*, Dec. 2023 (rev. 2024). https://www.aivd.nl/

[43] Australian Signals Directorate, “Planning for post-quantum cryptography” and *Information Security Manual*. https://www.cyber.gov.au/business-government/secure-design/quantum/planning-for-post-quantum-cryptography

[44] OWASP CycloneDX, *CycloneDX Specification v1.6: Cryptography Bill of Materials (CBOM)*, Apr. 2024. https://cyclonedx.org/capabilities/cbom/

[45] OpenSSL Project, “OpenSSL 3.5 release notes,” Apr. 2025. https://openssl-library.org/

[46] OpenJDK, “JEP 496: Quantum-Resistant Module-Lattice-Based Key Encapsulation Mechanism” and “JEP 497: Quantum-Resistant Module-Lattice-Based Digital Signature Algorithm,” JDK 24, 2025. https://openjdk.org/jeps/496 ; https://openjdk.org/jeps/497

[47] D. J. Bernstein et al., “KyberSlash: Exploiting secret-dependent division timings in Kyber implementations,” IACR ePrint 2024/1049, 2024. https://eprint.iacr.org/2024/1049

[48] J. B. Almeida et al., “Formally verifying Kyber, Episode IV: Implementation correctness,” *IACR TCHES*, vol. 2023, no. 3, pp. 253–282, 2023. doi:10.46586/tches.v2023.i3.253-282.

[49] W. Castryck and T. Decru, “An efficient key recovery attack on SIDH,” in *EUROCRYPT 2023*, LNCS 14008, pp. 423–447. doi:10.1007/978-3-031-30589-4_15.

[50] W. Beullens, “Breaking Rainbow takes a weekend on a laptop,” in *CRYPTO 2022*, LNCS 13508, pp. 464–479. doi:10.1007/978-3-031-15979-4_16.

[51] National Security Agency, “Quantum Key Distribution (QKD) and Quantum Cryptography (QC).” https://www.nsa.gov/Cybersecurity/Quantum-Key-Distribution-QKD-and-Quantum-Cryptography-QC/

[52] UK National Cyber Security Centre, “Quantum security technologies,” white paper, 2020. https://www.ncsc.gov.uk/whitepaper/quantum-security-technologies

Note on format: this paper is supplied as complete, typesetting-ready text (Markdown with tables). It can be converted directly to PDF (for example with Pandoc and LaTeX, using an IEEEtran template).